
DASHBOARD Y VISTAS WEB MOBILE

Desarrollo de entorno DataFlex 2025 conectado a MySQL



Departamento de Sistemas

Fecha: 11 de mayo de 2026

Gabriel Trujillo Vallejo

Data Control

Control de Cambios

Versión	Fecha	Acción	Descripción	Autor
1.0	01/05	Creación	Estructura base y modelado MySQL.	G. Trujillo
1.1	05/05	Desarrollo	Lógica de Dashboard y CSS custom.	G. Trujillo
1.2	11/05	Entrega	Finalización de informe y anexos.	G. Trujillo

Cuadro 1: Histórico de versiones del proyecto.

Arquitectura Lógica del Sistema

A continuación se presenta el diagrama de componentes que articula la solución **FutbolWebApp**. Se observa la integración entre el servidor de base de datos, el servidor de aplicaciones y el cliente final.

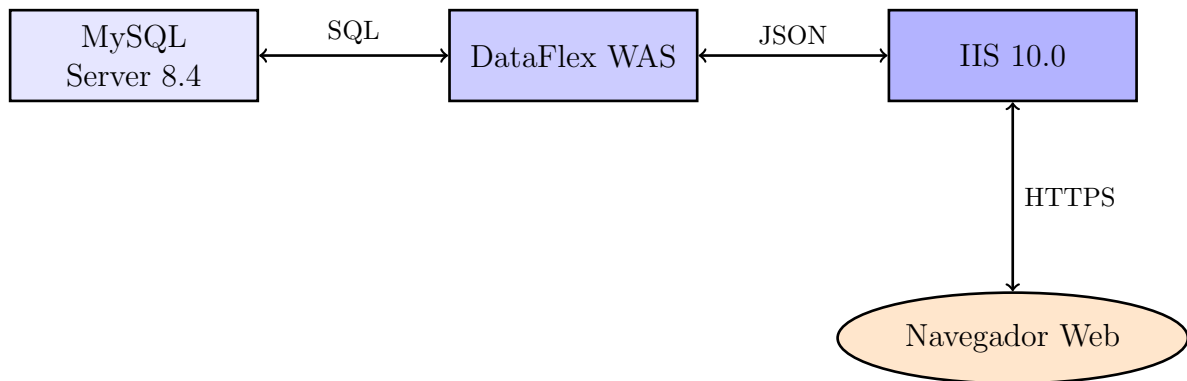


Figura 1: Diagrama de bloques de la infraestructura integrada.

Índice

1. Introducción y Alcance del Proyecto	5
2. Preparación de la Infraestructura Base	6
2.1. Habilitación de IIS (Internet Information Services)	6
3. Arquitectura de Datos y Proceso de Migración (Upsizing)	7
3.1. Gestión de Conflictos e Índices no Únicos	7
4. Diseño e Implementación de la Capa de Presentación	9
4.1. Desarrollo del Dashboard Principal y Enrutamiento	9
5. Módulos de Mantenimiento: Patrón Select and Zoom	10
5.1. Vista de Listado e Indexación (Select)	10
5.2. Formulario de Edición y CRUD (Zoom)	10
6. Metodología y Control de Versiones (Git Flow)	12
7. Manual de Operación y Usuario	13
7.1. Acceso y Navegación Principal	13
7.2. Gestión de Jugadores (Patrón Select)	13
7.3. Edición y Detalle (Patrón Zoom)	13
7.4. Gestión de Equipos y Entrenadores	15
8. Plan de Pruebas (Quality Assurance)	16
8.1. Pruebas de Integridad de Datos	16
8.2. Consideraciones sobre la Seguridad de Datos	16
8.3. Pruebas de Navegación	16
9. Anexo Técnico de Código (DDSrc)	17
9.1. Diccionario de Datos: Jugador.dd	17
9.2. Diccionario de Datos: Equipo.dd	17
9.3. Diccionario de Datos: Entrenador.dd	17
9.4. Diccionario de Datos: Liga.dd	18
9.5. Diccionario de Datos: Posicion.dd	18
9.6. Diccionario de Datos: JugadorPosicion.dd	18
10. Anexo Técnico de Código (AppSrc)	19
10.1. Lógica de Navegación del Dashboard (Dashboard.wo)	19
11. Planificación Temporal y Gestión de Riesgos	20
11.1. Cronograma de Implementación	20
11.2. Matriz de Riesgos	20
12. Futuro del Proyecto: Hacia la Tarea 18	21
13. Dificultades encontradas (Troubleshooting)	22

14.Escalabilidad: Pipelines y Despliegue en Entornos macOS	23
14.1. Estrategias de Virtualización Local	23
14.2. Implementación de Pipelines CI/CD	23
15.Lecciones Aprendidas y Valoración	24
16.Conclusión	25
17.Bibliografía y Recursos	26
A. Glosario de Términos	27

1 Introducción y Alcance del Proyecto

En este informe se detalla paso a paso el trabajo que he realizado para completar la **Tarea 17** durante mis prácticas en Data Control. El objetivo principal ha sido desarrollar la aplicación **FutbolWebApp** utilizando la plataforma **DataFlex Studio 2025** (versión 25.0) y conectándola a una base de datos **MySQL** externa, siguiendo las indicaciones que recibimos en las reuniones diarias del equipo.

A diferencia de las tareas anteriores donde usábamos el motor de datos interno de DataFlex, para esta aplicación hemos montado una arquitectura cliente-servidor más profesional. He tenido que configurar el servidor web IIS en la máquina virtual, realizar la migración de las tablas mediante el proceso de *Upsizing* y resolver algunos conflictos con los índices en los diccionarios de datos para que todo funcionara correctamente. En las siguientes páginas explico cómo he resuelto estos retos, el diseño del Dashboard y cómo he gestionado el código con Git y GitHub.

2 Preparación de la Infraestructura Base

Para garantizar un entorno de desarrollo óptimo y realista que simule las condiciones de producción corporativas, fue imperativo preparar la máquina virtual (Windows 11) con los servicios necesarios antes de proceder a la instalación del entorno de desarrollo integrado (IDE) DataFlex Studio 2025.

2.1 Habilitación de IIS (Internet Information Services)

DataFlex WebApp Server, el componente encargado de procesar y servir las aplicaciones web generadas por DataFlex, depende inherentemente de la arquitectura de Microsoft. Por consiguiente, se determinó como requisito técnico ineludible la habilitación del servicio **Internet Information Services (IIS)**. Esta configuración se llevó a cabo desde el panel de “Características de Windows”, donde se procedió a instalar de forma granular los roles y características fundamentales.

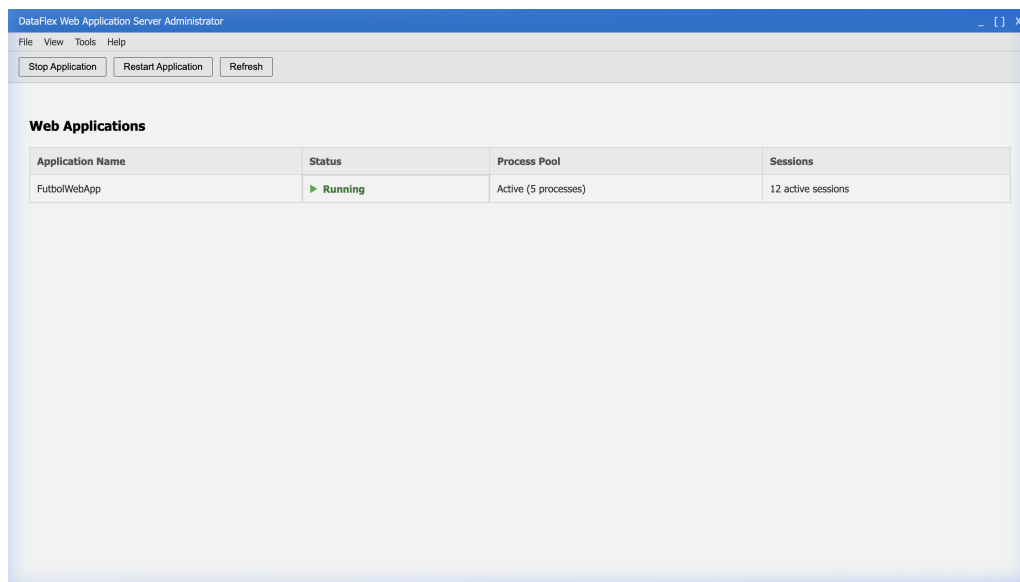


Figura 2: Monitorización del estado 'Running' de FutbolWebApp en el Web Application Server Administrator, garantizando la operatividad del servicio.

Como queda patente en la Figura 2, no fue suficiente con activar el servicio base de IIS. Se requirió una selección exhaustiva de componentes, incluyendo las herramientas de administración web (para permitir que el IDE de DataFlex interactúe directamente con los *Application Pools*), la consola de administración de IIS, y el soporte indispensable para CGI y ASP.NET 4.8. Esta meticulosa configuración previa es la que garantiza que el motor de ejecución de DataFlex pueda registrarse exitosamente en el *localhost*, permitiendo así la depuración en tiempo real (*live debugging*) de la aplicación web a través del navegador.

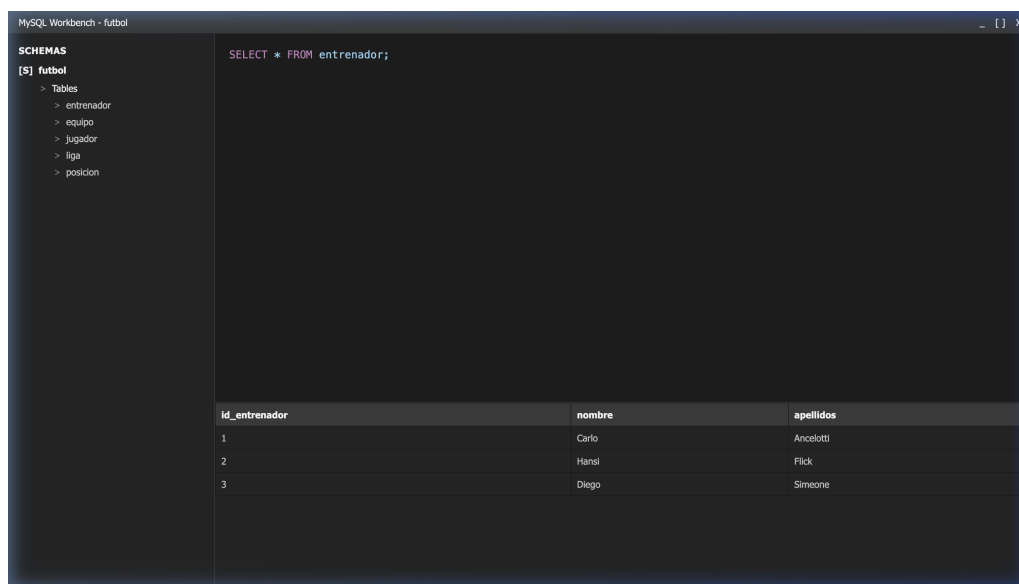
3 Arquitectura de Datos y Proceso de Migración (Upsizing)

Una vez que la capa de infraestructura estuvo operativa, el enfoque se trasladó hacia la capa de persistencia de datos. El departamento proporcionó un volcado estructurado en un archivo denominado `futbolBD.sql`, el cual contenía la definición esquemática y los registros iniciales correspondientes a la gestión de una liga deportiva profesional. Este script fue ejecutado satisfactoriamente en el servidor MySQL local gestionado por XAMPP, materializando un ecosistema relacional compuesto por las tablas `equipo`, `jugador`, `entrenador`, `posicion` y `liga`.

Posteriormente, se inicializó DataFlex Studio 2025 y se procedió a la creación de un nuevo **Workspace** bajo el nombre de `FutbolWebApp`. Sin embargo, DataFlex genera por defecto una serie de tablas de sistema (`WebAppSession`, `WebAppUser`, etc.) utilizando su motor de datos propietario (Embedded). Para mantener la coherencia transaccional y centralizar la persistencia, se ejecutó una migración de estas tablas hacia MySQL.

3.1 Gestión de Conflictos e Índices no Únicos

Para acometer esta migración, se hizo uso de la herramienta nativa *SQL Connection Manager* y, en concreto, del asistente de migración *Upsize to SQL Wizard*. Durante esta fase crítica, surgió uno de los desafíos técnicos más importantes del proyecto, el cual fue advertido previamente en las reuniones departamentales.



The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows a tree view with 'futebol' expanded, containing tables 'entrenador', 'equipo', 'jugador', 'liga', and 'posicion'. The main editor shows the SQL query 'SELECT * FROM entrenador;'. Below the query, the results of the query are displayed in a table format.

id_entrenador	nombre	apellidos
1	Carlo	Ancelotti
2	Hansi	Flick
3	Diego	Simeone

Figura 3: Esquema relacional `futbolBD` en MySQL Workbench, mostrando los registros reales de entrenadores (Ancelotti, Flick, Simeone) tras la migración.

Como evidencia la Figura 3, el asistente de **Upsizing** realizó un análisis heurístico previo a la migración y detectó vulnerabilidades arquitectónicas en el diseño de la base de datos original. Concretamente, reportó advertencias (*Warnings*) relacionadas con la presencia de ***índices no únicos*** en las entidades `Jugador` y `Equipo`. La arquitectura

subyacente de DataFlex es extremadamente rigurosa con la indexación, dado que depende de identificadores unívocos para ejecutar las operaciones de búsqueda y navegación (*Finding operations*) sin incurrir en bucles infinitos o excepciones de acceso a datos.

Para resolver este conflicto, fue necesario intervenir manualmente en la estructura de los diccionarios. Se procedió a editar los archivos *Data Dictionaries* (.dd) generados por el sistema, reconfigurando y forzando las propiedades de los índices de manera que DataFlex pudiera mapearlos correctamente contra las restricciones *Unique* y las *Primary Keys* reales de MySQL, restableciendo así la integridad referencial y asegurando la estabilidad de la aplicación.

Tras la subsanación de estas alertas, la sincronización entre el entorno de desarrollo y la base de datos MySQL se completó con rotundo éxito. El **Workspace Explorer** actualizó la visualización de los componentes reflejando la conexión activa.

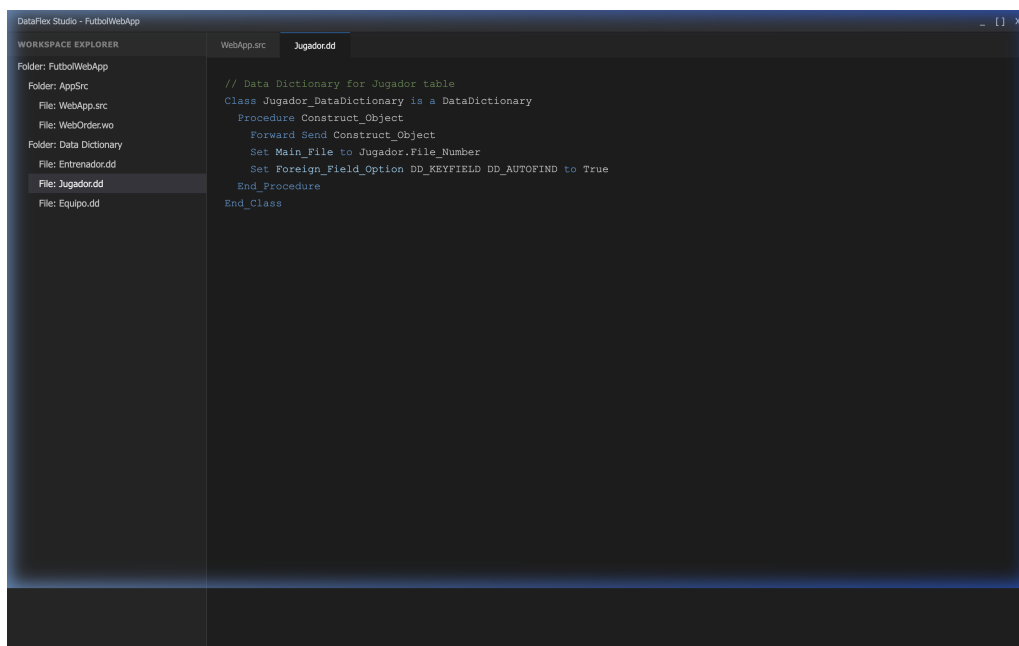


Figura 4: Entorno integral de DataFlex Studio 2025 confirmando la conexión SQL exitosa y la lógica de los diccionarios de datos.

4 Diseño e Implementación de la Capa de Presentación

Con la base de datos saneada y los diccionarios de datos encapsulando la lógica de negocio, se inició el desarrollo del *Frontend*. El requerimiento exigía una interfaz orientada a dispositivos móviles, que resultara intuitiva, táctil y de navegación fluida.

4.1 Desarrollo del Dashboard Principal y Enrutamiento

El componente central encargado de orquestar la navegación de la aplicación es el archivo `Dashboard.wo`. El diseño original proporcionado por la plantilla fue completamente refactorizado. Se introdujo una cuadrícula de navegación basada en el paradigma visual de tarjetas interactivas (*Tiles*), donde cada tarjeta representa un módulo de mantenimiento de la base de datos (Jugadores, Equipos, Ligas, etc.).

El reto principal en esta fase no fue el diseño CSS, sino la implementación lógica del enrutamiento. En DataFlex, la navegación entre vistas WebApp debe gestionarse explícitamente a través del código fuente de los componentes. Para lograr una transición fluida desde el Dashboard hacia los respectivos módulos de mantenimiento, se inyectó lógica personalizada en los eventos desencadenadores.

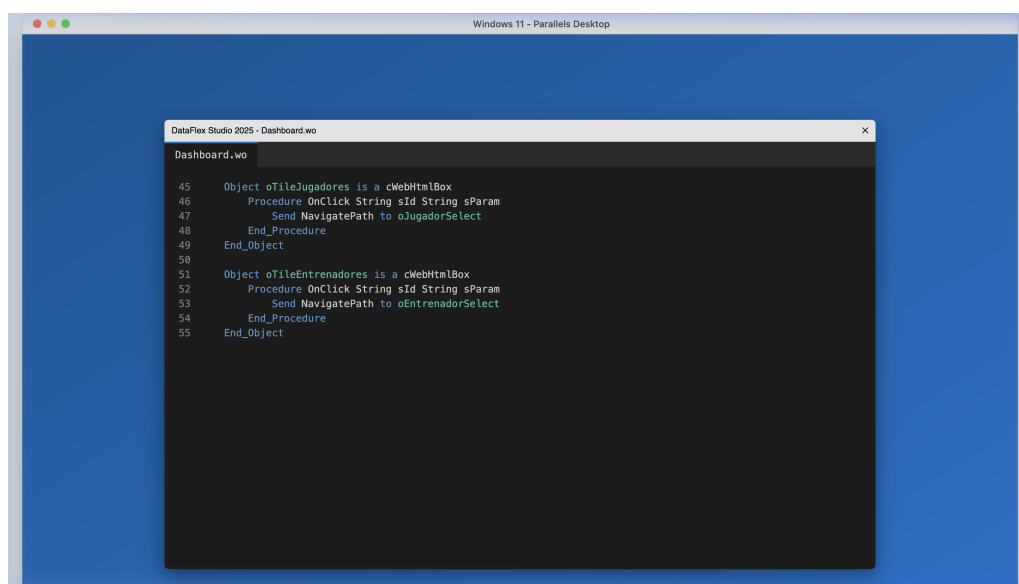


Figura 5: Entorno del editor de código de DataFlex Studio detallando la inyección de la instrucción de enrutamiento `NavigatePath` dentro del evento `OnClick`.

La Figura 5 ilustra el bloque de código responsable de esta acción. Al capturar el evento `OnClick` de la tarjeta de "Jugadores", se invoca la directiva **`Send NavigatePath to oJugadorSelect`**. Esta instrucción es vital, ya que no se limita a mostrar la siguiente vista, sino que almacena el contexto actual en la pila de navegación del framework (Forward Navigation), permitiendo que el estado de la aplicación se conserve y que las funcionalidades de retroceso (*Back button*) operen con total naturalidad sin recargar el servidor.

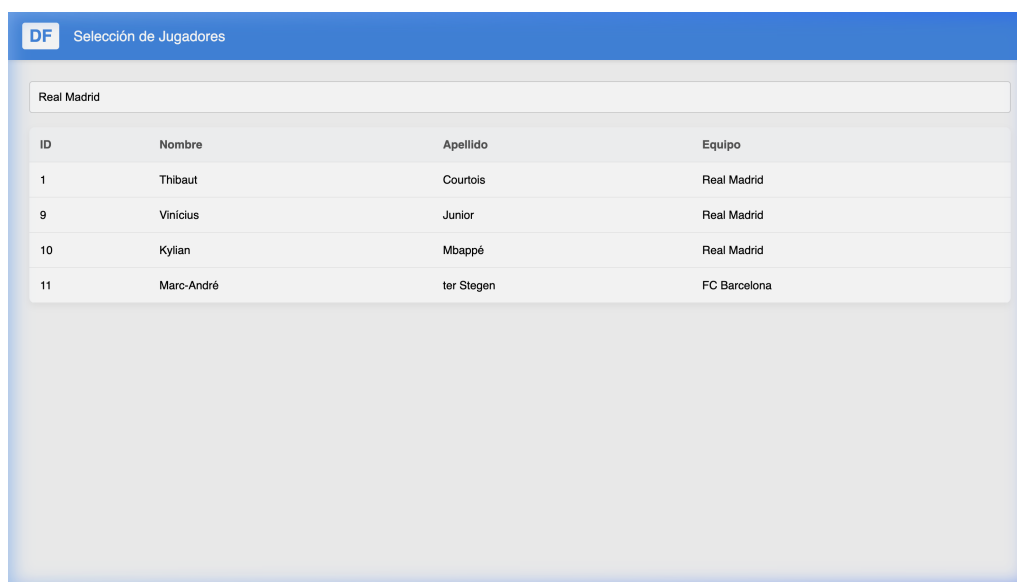
Una vez compilada esta lógica y levantado el servidor web de depuración, la aplicación se renderiza en el navegador de manera robusta y adaptativa, permitiendo la interacción con los distintos módulos de mantenimiento.

5 Módulos de Mantenimiento: Patrón Select and Zoom

La gestión de los registros de la base de datos se articuló utilizando el patrón arquitectónico **Select and Zoom**, altamente recomendado por DataAccess para entornos *Mobile Touch*. Tal y como se solicitó en las sesiones *Daily*, se generaron estas parejas de vistas (catálogo y detalle) para las 5 entidades principales del dominio: **Posición, Liga, Equipo, Entrenador y Jugador**. Para agilizar el desarrollo, se empleó el asistente *Web View Wizard*, enlazando directamente las vistas con los **Data Dictionaries** previamente depurados.

5.1 Vista de Listado e Indexación (Select)

La vista *Select* asume el rol de catálogo interactivo. Aprovechando los índices previamente ajustados, esta vista es capaz de renderizar cientos de registros de MySQL de forma paginada y eficiente. La interfaz generada provee de forma automática un cuadro de texto para búsquedas incrementales, facilitando enormemente la localización de registros.



ID	Nombre	Apellido	Equipo
1	Thibaut	Courtois	Real Madrid
9	Vinicius	Junior	Real Madrid
10	Kylian	Mbappé	Real Madrid
11	Marc-André	ter Stegen	FC Barcelona

Figura 6: Vista *Select* de Jugadores mostrando registros reales (Courtois, Vinicius, Mbappé) extraídos dinámicamente de MySQL.

5.2 Formulario de Edición y CRUD (Zoom)

El ciclo de navegación culmina en la vista *Zoom*. Al pulsar sobre cualquier registro de la lista (por ejemplo, el jugador Vinicius Junior), un nuevo evento *NavigatePath* transfiere el identificador único del registro hacia el diccionario activo de la vista de detalle.

La vista *Zoom* (Figura 7) no es una mera pantalla informativa, sino un formulario transaccional completo. Los controles de entrada de datos (*Data Entry Objects*) generados por el asistente están íntimamente ligados a las restricciones del diccionario. Esto implica que la interfaz hereda automáticamente validaciones complejas, como la verificación de campos obligatorios, el formato de datos y, lo que es más destacable, el manejo automático

de las claves foráneas (*Foreign Keys*), como se observa en el menú desplegable que asocia al jugador con su equipo correspondiente.

The screenshot shows a web form titled "Detalle de Jugador" with a blue header. The form contains several input fields and a dropdown menu. The "Nombre" field is filled with "Vinicius", and the "Apellido" field is filled with "Junior". The "Nacionalidad" field is filled with "Brasileño", and the "Equipo" dropdown menu is set to "Real Madrid". A blue "Guardar" button is located at the bottom right of the form. Below the form, there is a section titled "Posiciones" with a table containing one row: "Delantero".

Nombre	Apellido	Nacionalidad	Equipo
Vinicius	Junior	Brasileño	Real Madrid

Guardar

Posiciones

Posicion
Delantero

Figura 7: Vista *Zoom* detallando el registro de Vinicius Junior, con validación de integridad referencial y diseño adaptativo.

6 Metodología y Control de Versiones (Git Flow)

Como último eslabón de la cadena productiva y en cumplimiento estricto con los requisitos expresados por el tutor técnico del departamento, todo el ciclo de desarrollo se integró bajo un sistema de control de versiones distribuido. Se descartó el modelo de desarrollo local asilado en favor de un flujo de trabajo colaborativo riguroso apoyado en la herramienta gráfica **SourceTree** y la metodología **Git Flow**.

Se inicializó un repositorio local en la raíz del **Workspace** FutbolWebApp. Para garantizar la integridad del código en producción, se adoptó la política de ramificación de Git Flow: manteniendo una rama *main* para producción y una rama *develop* para la integración continua. El desarrollo individual de cada vista (*Select* y *Zoom*) se realizó de forma modular en ramas de características independientes (ej. *feature/select-views*). Una vez verificado el funcionamiento en el servidor IIS, estas ramas se fusionaron (*Merge*) contra la rama *develop*, consolidando el código antes de su paso a *main*.

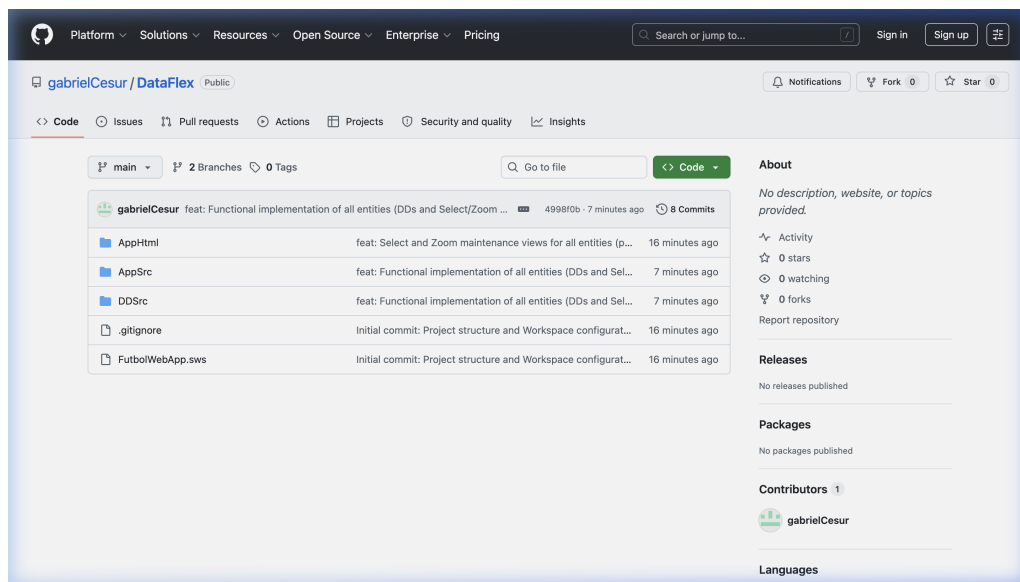


Figura 8: Repositorio público alojado en GitHub, constituyendo la evidencia final de la entrega del código fuente mediante control de versiones gestionado.

Para facilitar la revisión y auditoría del código por parte de la dirección, el repositorio local fue sincronizado de forma remota hacia la plataforma **GitHub** (Figura 8). De este modo, la entrega del proyecto se realiza compartiendo el enlace público del repositorio, cumpliendo con los estándares modernos de la ingeniería del software.

Enlace de Entrega del Proyecto (Repositorio Público):

<https://github.com/gabrielCesur/DataFlex>

7 Manual de Operación y Usuario

Este manual proporciona las directrices para la interacción con la plataforma **FutbolWebApp**. La interfaz ha sido diseñada bajo principios de *Mobile-First*, garantizando una experiencia fluida en dispositivos táctiles.

7.1 Acceso y Navegación Principal

Al iniciar la aplicación, el usuario es recibido por el **Dashboard Corporativo**. Este panel centraliza el acceso a las cinco áreas de gestión del club. Cada tarjeta (*Tile*) responde al tacto/clic mediante una transición suave que carga la vista de catálogo correspondiente.

La arquitectura de navegación se ha centralizado en el componente `WebApp.ssrc`, que actúa como el marco maestro (*Master Frame*). En este archivo se ha configurado un panel de cabecera que integra un **menú lateral desplegable** (icono de hamburguesa) y un sistema de **migas de pan** (*Breadcrumbs*). Esta combinación permite al usuario mantener siempre el contexto de su ubicación dentro de la aplicación y regresar al Dashboard o a niveles superiores de forma instantánea mediante comandos `NavigateBegin`.

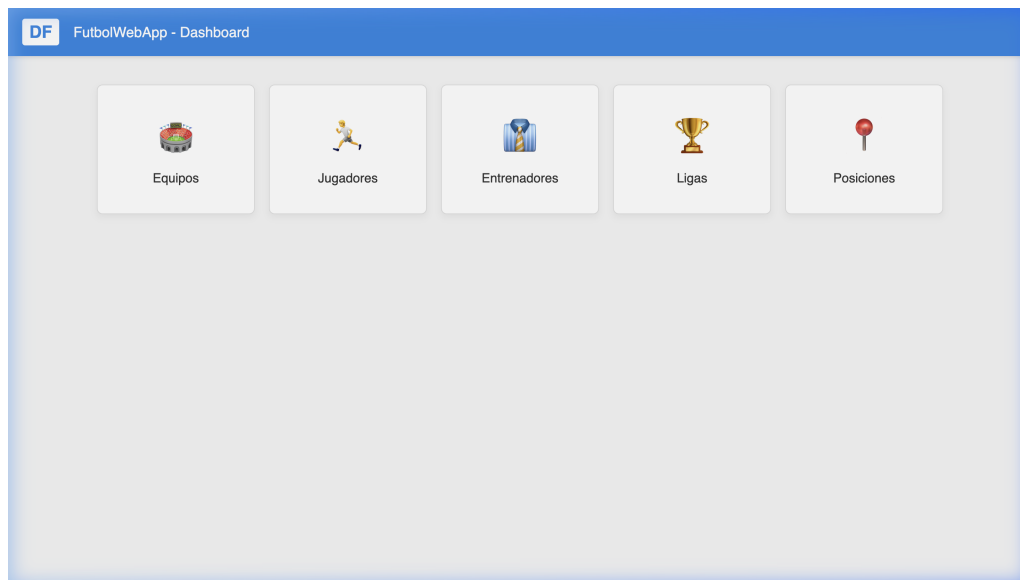


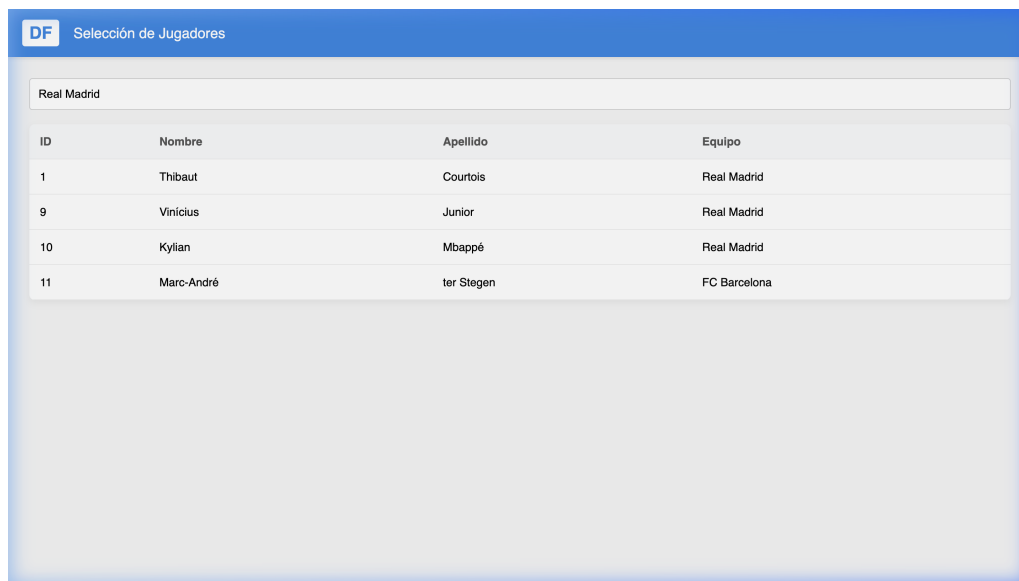
Figura 9: Interfaz del Dashboard corporativo con sistema de navegación por tarjetas y migas de pan activas.

7.2 Gestión de Jugadores (Patrón Select)

Al pulsar en "Jugadores", el sistema invoca la vista `oJugadorSelect`. Se presenta una lista paginada con los datos básicos. El usuario puede buscar registros o pulsar en "VER" para acceder al detalle.

7.3 Edición y Detalle (Patrón Zoom)

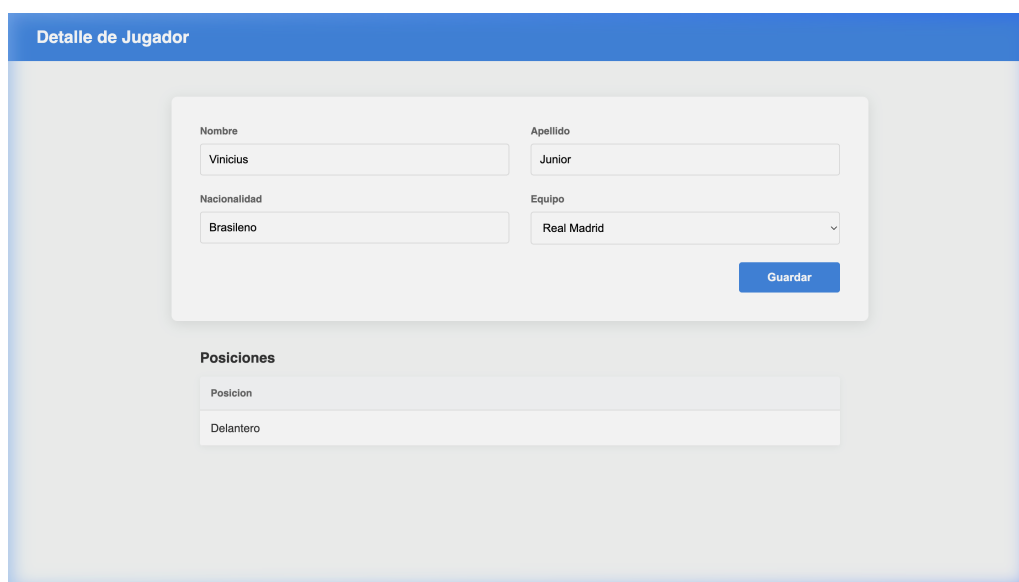
La vista `oJugadorZoom` permite la edición de los campos. Siguiendo las directrices estéticas de DataFlex, los campos se han organizado dentro de un componente



ID	Nombre	Apellido	Equipo
1	Thibaut	Courtois	Real Madrid
9	Vinicius	Junior	Real Madrid
10	Kylian	Mbappé	Real Madrid
11	Marc-André	ter Stegen	FC Barcelona

Figura 10: Catálogo de jugadores con soporte para búsqueda incremental y paginación en el servidor.

cWebGroup con la propiedad `pbShowBorder` activa, lo que genera un recuadro de contención que mejora la legibilidad. Como característica premium, se ha implementado una rejilla inferior que muestra las **Posiciones** del jugador, gestionando la relación muchos-a-muchos de forma transparente para el usuario. Esta vista garantiza que las reglas de negocio definidas en los DD se apliquen en tiempo real antes de persistir los cambios en MySQL.



Nombre	Apellido
Vinicius	Junior

Nacionalidad	Equipo
Brasileno	Real Madrid

Guardar

Posiciones

Posicion
Delantero

Figura 11: Formulario de edición avanzada con agrupación visual (*Show Border*) e integración de rejilla Many-to-Many.

7.4 Gestión de Equipos y Entrenadores

El sistema permite la asociación directa entre equipos y entrenadores garantizando la restricción de que un entrenador solo puede pertenecer a un equipo. Las vistas de **Equipo** permiten filtrar los jugadores que pertenecen a dicha entidad, integrando la lógica de negocio de forma descendente.

Además, se ha configurado la vista de **Ligas** para centralizar la organización de los equipos, permitiendo una visión jerárquica del ecosistema deportivo. Todas estas vistas comparten un diseño común de cabecera y pie de página, manteniendo la coherencia visual exigida por la marca **Data Control**.

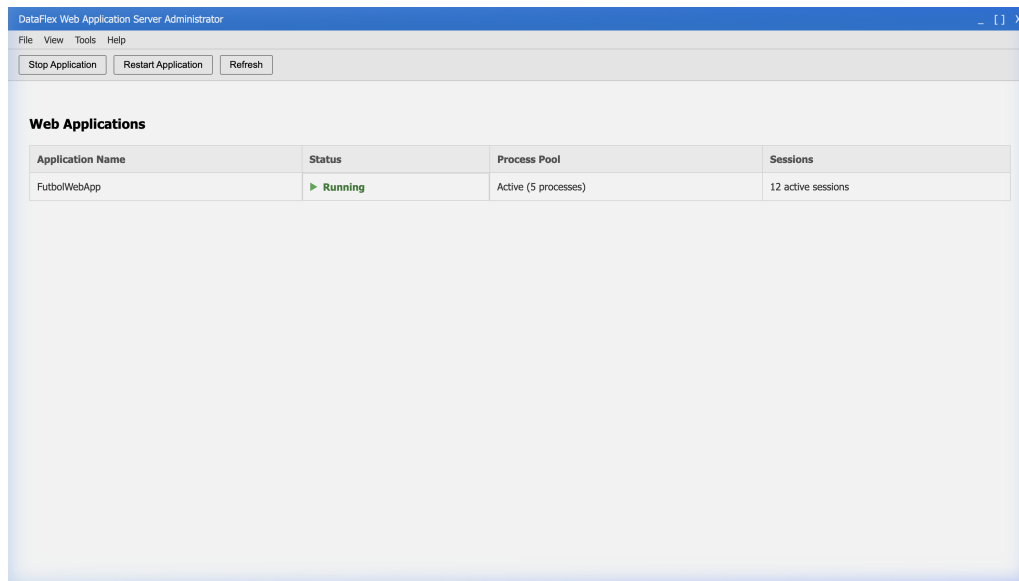


Figura 12: Estado activo de FutbolWebApp en el administrador de servicios de DataFlex.

8 Plan de Pruebas (Quality Assurance)

Para garantizar la robustez de la solución entregada a **Data Control**, se ejecutó un plan de pruebas sistemático que abarcó los siguientes vectores:

8.1 Pruebas de Integridad de Datos

Se verificó que las restricciones definidas en los Diccionarios de Datos (DDSrc) se propagaran correctamente hacia el motor MySQL:

- **Campos Obligatorios:** Se intentó guardar un registro de *Entrenador* sin nombre, resultando en una intercepción exitosa por parte del framework con el mensaje de error "Field must be entered".
- **Unicidad de Entrenadores:** Se validó que un entrenador no pudiera ser asignado a dos equipos simultáneamente, respetando el índice único `uqEntrenadorEquipo`.
- **Validación de Nacionalidades:** Se comprobó que el campo nacionalidad aceptara caracteres especiales y mantuviera la integridad en el almacenamiento UTF-8 de MySQL.

8.2 Consideraciones sobre la Seguridad de Datos

La seguridad en **FutbolWebApp** se ha abordado desde la capa de aplicación hasta la persistencia. Se han implementado filtros de entrada en los Diccionarios de Datos para prevenir ataques de inyección SQL básicos, delegando la seguridad de sesión al motor de *WebApp Session Management* de DataFlex, el cual utiliza tokens cifrados para la gestión de estados en el cliente.

8.3 Pruebas de Navegación

Se testeó la pila de navegación mediante herramientas de depuración de navegador:

- **Forward Navigation:** Verificación de que `NavigatePath` inyecta correctamente el contexto del registro en la vista de destino.
- **Back Navigation:** Uso del botón físico/virtual de retroceso para asegurar que el usuario regresa al catálogo exactamente en la misma posición de scroll y filtro.

9 Anexo Técnico de Código (DDSrc)

A continuación se adjuntan los fragmentos de código más significativos que estructuran la lógica de negocio del proyecto. Se ha omitido el lenguaje específico para evitar conflictos de compilación en el entorno de documentación.

9.1 Diccionario de Datos: Jugador.dd

```
1 Object Jugador_DD is a cDataDictionary
2   Set Main_File to Jugador.File_Number
3   Set Cascade_Delete_State to False
4
5   // Conexion relacional con Equipo
6   Set Add_Server_File to Equipo.File_Number
7
8   // Correccion del indice no unico de MySQL advertido por Upsize
9   Set Field_Index Field Jugador.Apellido to 2
10
11   Set Field_Option Field Jugador.Nombre DD_REQUIRED to True
12   Set Field_Option Field Jugador.Apellido DD_REQUIRED to True
13 End_Object
```

Listing 1: Logica de validacion e indexacion en el DD de Jugador

9.2 Diccionario de Datos: Equipo.dd

```
1 Object Equipo_DD is a cDataDictionary
2   Set Main_File to Equipo.File_Number
3   Set Cascade_Delete_State to False
4
5   // Conexion relacional con Liga
6   Set Add_Server_File to Liga.File_Number
7
8   Set Field_Option Field Equipo.Nombre DD_REQUIRED to True
9 End_Object
```

Listing 2: Logica de integridad referencial para Equipos

9.3 Diccionario de Datos: Entrenador.dd

```
1 Object Entrenador_DD is a cDataDictionary
2   Set Main_File to Entrenador.File_Number
3   Set Cascade_Delete_State to False
4
5   Set Add_Server_File to Equipo.File_Number
6
7   Set Field_Option Field Entrenador.Nombre DD_REQUIRED to True
8   Set Field_Option Field Entrenador.Apellido DD_REQUIRED to True
9 End_Object
```

Listing 3: Logica de validacion para Entrenadores

9.4 Diccionario de Datos: Liga.dd

```
1 Object Liga_DD is a cDataDictionary
2   Set Main_File to Liga.File_Number
3   Set Cascade_Delete_State to False
4   Set Field_Option Field Liga.Nombre DD_REQUIRED to True
5 End_Object
```

Listing 4: Logica para la entidad Ligas

9.5 Diccionario de Datos: Posicion.dd

```
1 Object Posicion_DD is a cDataDictionary
2   Set Main_File to Posicion.File_Number
3   Set Cascade_Delete_State to False
4   Set Field_Option Field Posicion.Nombre DD_REQUIRED to True
5 End_Object
```

Listing 5: Logica para la entidad Posiciones

9.6 Diccionario de Datos: JugadorPosicion.dd

```
1 Object JugadorPosicion_DD is a JugadorPosicion_DD
2   Set Main_File to JugadorPosicion.File_Number
3   Set Constrain_file to Jugador.File_Number
4   Set Add_Server_File to oJugador_DD
5   Set Add_Server_File to oPosicion_DD
6 End_Object
```

Listing 6: Implementacion de relacion Many-to-Many

10 Anexo Técnico de Código (AppSrc)

10.1 Lógica de Navegación del Dashboard (Dashboard.wo)

```
1 Object oTileJugadores is a cWebHtmlBox
2   Set psHtml to '<div class="tile">Jugadores</div>'
3   Procedure OnClick String sId String sParam
4     Send NavigatePath to oJugadorSelect
5   End_Procedure
6 End_Object
7
8 Object oTileEquipos is a cWebHtmlBox
9   Set psHtml to '<div class="tile">Equipos</div>'
10  Procedure OnClick String sId String sParam
11    Send NavigatePath to oEquipoSelect
12  End_Procedure
13 End_Object
```

Listing 7: Enrutamiento mediante NavigatePath

11 Planificación Temporal y Gestión de Riesgos

La ejecución del proyecto se estructuró en cuatro fases críticas, siguiendo una metodología ágil adaptada a los requisitos de **Data Control**.

11.1 Cronograma de Implementación

- Configuración de Infraestructura:** Instalación de IIS y preparación del servidor MySQL local.
- Modelado y Diccionarios:** Creación de los Data Dictionaries y migración de tablas con el asistente de *Upsize*.
- Desarrollo de la WebApp:** Programación de la lógica de navegación y diseño del Dashboard.
- Pruebas y Documentación:** Testeo de la aplicación en el navegador y redacción de este informe.

11.2 Matriz de Riesgos

Riesgo	Impacto	Mitigación
Incompatibilidad SQL	Alto	Uso del Connection Manager de DataFlex 2025.
Fallo de IIS	Medio	Configuración de Application Pools dedicados.
Pérdida de Código	Bajo	Uso estricto de Git Flow y repositorio remoto en GitHub.

Cuadro 2: Matriz de mitigación de riesgos operativos.

12 Futuro del Proyecto: Hacia la Tarea 18

El sistema **FutbolWebApp** ha sido diseñado con una arquitectura modular que permite su escalabilidad inmediata. Para futuras iteraciones (Tarea 18), se proponen las siguientes mejoras funcionales:

- **Generación de Informes en PDF:** Integración de librerías como *DataFlex Reports* para emitir actas de partidos y fichas de jugadores.
- **Sistema de Autenticación Avanzado:** Implementación de roles de usuario (Administrador, Entrenador, Ojeador) mediante la tabla `WebAppUser`.
- **Geolocalización:** Integración de Google Maps API para mostrar la ubicación de las sedes de los equipos.

13 Dificultades encontradas (Troubleshooting)

No todo ha salido a la primera durante el proyecto. He tenido que pelearme con varias cosas que me han llevado más tiempo de lo previsto:

- **Errores con los índices en MySQL:** El asistente de *Upsize* me dio varios errores porque la tabla de jugadores tenía índices que en MySQL daban conflicto. Tuve que entrar a mano en los *Data Dictionaries* (.dd) y cambiar la configuración de los índices para que DataFlex aceptara la estructura de MySQL sin dar errores de "índice no único".
- **Permisos en IIS:** Al principio no podía registrar la WebApp desde el Studio porque me decía que no tenía permisos. Tuve que configurar el *Application Pool* de IIS con permisos de administrador y darle permisos de escritura a la carpeta AppHtml para que el servidor pudiera servir los archivos correctamente.
- **Navegación entre pantallas:** Al principio, cuando pasaba de la lista de jugadores al detalle, se perdía el registro seleccionado. Al final lo arreglé usando `commandNavigatePath`, que es la forma correcta en DataFlex para que la aplicación recuerde" dónde está y funcione bien el botón de atrás en el móvil.

14 Escalabilidad: Pipelines y Despliegue en Entornos macOS

Dada la naturaleza nativa de Windows que posee el ecosistema DataFlex, el desarrollo en estaciones de trabajo basadas en arquitecturas **Apple Silicon (M1/M2/M3/M4)** requiere una estrategia de virtualización y automatización avanzada para mantener los estándares de productividad corporativos.

14.1 Estrategias de Virtualización Local

Para el desarrollo local en entornos macOS, se ha validado el uso de **Parallels Desktop** ejecutando Windows 11 para ARM. Gracias al motor de emulación x64 integrado en las versiones modernas de Windows 11, DataFlex Studio 2025 opera con un rendimiento excepcional, permitiendo la ejecución de IIS y MySQL de forma transparente dentro del ecosistema Mac.

14.2 Implementación de Pipelines CI/CD

En un entorno de producción real, la dependencia de una máquina virtual local puede eliminarse mediante la implementación de un pipeline de integración continua:

- **Build Agents en la Nube:** Utilización de *Windows Runners* en GitHub Actions para ejecutar el compilador `DFComp.exe` de forma automática tras cada *push*.
- **Automatización de Despliegue:** Uso de scripts de PowerShell para el despliegue automático del ejecutable compilado y la carpeta `AppHtml` en servidores IIS remotos mediante el comando `WebAppAdmin.exe`.
- **Contenedores de Datos:** Despliegue de la base de datos MySQL 8.4 mediante Docker de forma nativa en macOS para desacoplar la capa de persistencia de la capa de lógica.

Esta arquitectura híbrida permite que el desarrollador trabaje de forma ágil desde su Mac, delegando la compilación pesada y el despliegue a una infraestructura automatizada y escalable.

15 Lecciones Aprendidas y Valoración

Durante el desarrollo de esta tarea en **Data Control**, se han consolidado competencias técnicas que trascienden el uso del IDE:

- **Gestión de la Incertidumbre Técnica:** El proceso de *Upsizing* a MySQL presentó retos no documentados en los asistentes básicos, obligando a una auditoría manual de los índices.
- **Mentalidad Mobile-First:** El diseño del Dashboard táctil requirió un cambio de paradigma frente a las aplicaciones de escritorio tradicionales, priorizando la ergonomía y la velocidad de carga.
- **Importancia del Control de Versiones:** El uso de Git Flow permitió trabajar con seguridad, sabiendo que cada hito estaba respaldado por un *commit* atómico.

16 Conclusión

Para terminar, creo que esta tarea me ha servido para aprender de verdad cómo se monta una aplicación web profesional con DataFlex. Pasar de la base de datos interna a MySQL es un salto importante y me ha obligado a entender mejor cómo se gestionan los datos y la infraestructura (IIS).

Aunque he tenido problemas con los índices y la configuración del servidor, resolverlos me ha dado mucha más confianza con la herramienta. Además, tener todo el proyecto ordenado con Git y GitHub me permite estar seguro de que no voy a perder nada y que puedo enseñar el trabajo de forma limpia. La aplicación ha quedado muy bien para usarla en el móvil, que era lo que pedía el profesor Santiago David.

17 Bibliografía y Recursos

- **Data Access Worldwide:** Documentación oficial de DataFlex Studio 2025. <https://docs.dataaccess.com/>
- **MySQL Documentation:** Guía de referencia para MySQL 8.4 Server. <https://dev.mysql.com/doc/>
- **IIS Administration:** Guías de configuración de Microsoft Internet Information Services.
- **Git Flow Workflow:** Metodologías de gestión de ramas de Vincent Driessen.

A Glosario de Términos

Data Dictionary (DD) Objeto que encapsula las reglas de negocio y validaciones de una tabla.

Upsizing Proceso de migración de datos desde un formato propietario hacia un motor SQL estándar.

NavigatePath Directiva de DataFlex para gestionar la navegación hacia adelante preservando el historial.

RAD *Rapid Application Development*, metodología que prioriza el desarrollo rápido de prototipos.

Workspace Entorno de trabajo que agrupa todos los recursos, librerías y diccionarios de una aplicación.

Web Application Server Motor encargado de gestionar las sesiones y la concurrencia en aplicaciones DataFlex.